

FULL LEGAL NAME	LOCATION (COUNTRY)	EMAIL ADDRESS	MARK X FOR ANY NON-CONTRIBUTING MEMBER
Bianka Angelova Toneva	Bulgaria	bibitoneva@gmail.com	
Dipanshu Sharma	India	dipanshusharma.iitb@gmail.com	
Miklos Lammel	Hungary	lammelm@gmail.com	

Statement of integrity: By typing the names of all group members in the text boxes below, you confirm that the assignment submitted is original work produced by the group (excluding any non-contributing members identified with an “X” above).

Team member 1	Bianka Toneva
Team member 2	Dipanshu Sharma
Team member 3	Miklos Lammel

Use the box below to explain any attempts to reach out to a non-contributing member. Type (N/A) if all members contributed.

Note: You may be required to provide proof of your outreach to non-contributing members upon request.

(N/A)

Note: Please refer to the Python notebook for executable code, extensive commentary, and results for each question in this document.

Step 1 - Pseudocode For Hidden Markov Model Algorithms

Hidden Markov Models (HMMs) are used to model systems where the underlying states are not directly observable. Only the observable output states - a.k.a emission - are observed. The initial distribution over states is typically not observed, but assumed.

In this chapter, we show the construction and usage of the following HMM algorithms:

- Forward and backward algorithms are used to solve the problem of computing the probability of an observable sequence. - Evaluation
- The Viterbi algorithm decodes the most likely sequence of hidden states (state-transition path) for a given observed sequence. - Decoding
- Baum-Welch Algorithm, which is an Expectation Maximization (EM) algorithm that allows us to learn the HMM parameters from data. - Training

Forward Algorithm Pseudo Code

The forward algorithm computes the probability of an observed sequence given an HMM by recursively accumulating probabilities forward through time. This solves the evaluation problem: Given model parameters $\lambda=(\Pi,A,B)$ and an observed sequence $O=(o_1,\dots,o_T)$, compute $P(O|\lambda)$. This is the probability that λ produced O , and this is important because it lets us compare multiple models, or if we want to quantify how well our model explains a given sequence.

We decompose the problem of summing over all possible hidden state sequences (which would be computationally extremely expensive - exponential in T) into a recursive problem using the quantity called alpha.

Input:

N: Number of possible hidden states in the HMM

T: Length of the observed sequence

O: Sequence of observations ($o_1, \dots, o_T$)

A: State transition probability matrix, where $a_{ij} = P(q_t = j | q_{t-1} = i)$

B: Emission probability matrix, where $b_j(o_t) = P(\text{observation } o_t | q_t = j)$.

Π : Initial state distribution, where $\Pi[i] = P(q_1 = i)$

Step 1: Initialization ($t = 1$)

For each state i ($1 \leq i \leq N$):

$$\alpha_1(i) = \pi[i] * B[i](o_1)$$

Here, we compute the initial probabilities of starting in state i and generating the first observation of o_1 . And this is the base of the recursion.

Step 2: Induction ($2 \leq t \leq T$)

For each $t = 2$ to T :

For each state j ($1 \leq j \leq N$):

$$\alpha_t(j) = \sum_{i=1}^N [\alpha_{t-1}(i) * A[i][j]] * B[j](o_t)$$

For each timestep t from 2 to T , we calculate the probability that:

- We came from any previous state i
- Transitioned to the current state j
- And emitted the current observation o_t from state j

Sum over i handles marginalizing all the possible previous paths that resulted in the state j at time t .

Step 3: Termination

$$P(O | \lambda) = \sum_{i=1}^N \alpha_T(i)$$

At the time step T (final step), we sum the probabilities of being in any state at time T , and this generates the full observation sequence for us.

This method of summing over all possible final states i gives the total probability of the observation sequence O under model λ .

[remainder of the page intentionally left blank]

Backward Algorithm Pseudo Code

Input: Same as for the forward algorithm

Step 1: Initialization ($t = T$)

For each state i ($1 \leq i \leq N$):

$$\text{beta}_T(i) = 1$$

Step 2: Induction ($t = T-1$ down to 1)

For each $t = T-1$ down to 1:

For each state i ($1 \leq i \leq N$):

$$\text{beta}_t(i) = \sum_{j=1}^N [A[i][j] * B[j](o_{t+1}) * \text{beta}_{t+1}(j)]$$

Step 3: Termination

$$P(O \mid \lambda) = \sum_{i=1}^N [\pi[i] * B[i](o_1) * \text{beta}_1(i)]$$

Toy example for Forward and Backward algorithms

Given the observation that someone is walking, shopping, or napping (and considering the transition probabilities between Depressed and Happy states), we would like to infer what the underlying condition of the person was.

States: $S = \{\text{Depressed (1), Happy (2)}\}$, so

Number of states: $N = 2$.

Observations: $O = \{\text{walk, shop, nap}\}$

Transition matrix A =

	Depressed	Happy
Depressed	0.7	0.3
Happy	0.4	0.6

Table 1.1: Transmission matrix for all toy examples

Emission matrix B =

	walk	shop	nap
Depressed	0.1	0.4	0.5
Happy	0.6	0.3	0.1

Table 1.2: Emission matrix for all toy examples

Initial state distribution: π_i = [0.6 (Depressed), 0.4 (Happy)]

Observed sequence: O = [walk, shop, nap]

Number of timesteps: T = 3

Forward algorithm part

Initialization

$$\alpha_1(\text{Depressed}) = 0.6 \times 0.1 = 0.06$$

$$\alpha_1(\text{Sunny}) = 0.4 \times 0.6 = 0.24$$

Induction ($t=2$, $o_2 = \text{shop}$)

$$\alpha_2(\text{Depressed}) = [0.06 \times 0.7 + 0.24 \times 0.4] \times 0.4 = [0.042 + 0.096] \times 0.4 = 0.138 \times 0.4 = 0.0552$$

$$\alpha_2(\text{Happy}) = [0.06 \times 0.3 + 0.24 \times 0.6] \times 0.3 = [0.018 + 0.144] \times 0.3 = 0.162 \times 0.3 = 0.0486$$

Induction ($t=3$, $o_3 = \text{nap}$)

$$\alpha_3(\text{Depressed}) = [0.0552 \times 0.7 + 0.0486 \times 0.4] \times 0.5 = [0.03864 + 0.01944] \times 0.5 = 0.05808 \times 0.5 = 0.02904$$

$$\alpha_3(\text{Happy}) = [0.0552 \times 0.3 + 0.0486 \times 0.6] \times 0.1 = [0.01656 + 0.02916] \times 0.1 = 0.04572 \times 0.1 = 0.004572$$

Termination

$$P(O|\lambda) = \alpha_3(\text{Depressed}) + \alpha_3(\text{Happy}) = 0.02904 + 0.004572 = \mathbf{0.033612}$$

Backward algorithm part

Initialization

$$\beta_3(\text{Depressed}) = \beta_3(\text{Happy}) = 1$$

Induction $t = 2$

$$\beta_2(\text{Depressed}) = 0.7 \times 0.5 \times 1 + 0.3 \times 0.1 \times 1 = 0.35 + 0.03 = 0.38$$

$$\beta_2(\text{Happy}) = 0.4 \times 0.5 \times 1 + 0.6 \times 0.1 \times 1 = 0.2 + 0.06 = 0.26$$

Induction $t = 1$

$$\beta_1(\text{Depressed}) = 0.7 \times 0.4 \times 0.38 + 0.3 \times 0.3 \times 0.26 = [0.7 \times 0.4 \times 0.38] + [0.3 \times 0.3 \times 0.26]$$

$$\text{First term: } 0.7 \times 0.4 = 0.28, 0.28 \times 0.38 = 0.1064$$

$$\text{Second term: } 0.3 \times 0.3 = 0.09, 0.09 \times 0.26 = 0.0234$$

$$\text{Sum: } 0.1064 + 0.0234 = 0.1298$$

$$\beta_1(\text{Happy}) = 0.4 \times 0.4 \times 0.38 + 0.6 \times 0.3 \times 0.26 = [0.4 \times 0.4 \times 0.38] + [0.6 \times 0.3 \times 0.26]$$

$$\text{First term: } 0.4 \times 0.4 = 0.16, 0.16 \times 0.38 = 0.0608$$

$$\text{Second term: } 0.6 \times 0.3 = 0.18, 0.18 \times 0.26 = 0.0468$$

$$\text{Sum: } 0.0608 + 0.0468 = 0.1076$$

Termination

$$P(O|\lambda) = \sum_{i=1}^2 P_i(i) \times B_i \times \beta_1(i)$$

$$\text{Depressed: } 0.6 \times 0.1 \times 0.1298 = 0.007788$$

$$\text{Happy: } 0.4 \times 0.6 \times 0.1076 = 0.025824$$

$$\text{Total: } 0.007788 + 0.025824 = \underline{\underline{0.033612}}$$

Both algorithms yield the same results.

Interpretation of the results: $P(O|\lambda) = 0.0336$ quantifies that there is a 3.36% probability that we observe a person first **walking**, then **shopping**, then **napping** across three time periods, if their behaviors are controlled by the transmission and emission matrices' probabilities.

[remainder of the page intentionally left blank]

Viterbi Algorithm Pseudo Code

The Viterbi algorithm is a dynamic programming technique that can be used in HMMs to find the most probable sequence of hidden states (decoding) that produces a certain sequence of observations. The forward Viterbi computes the likely path from start to end. The backward Viterbi algorithm reconstructs the path from the end to the beginning after the optimal probability table has been filled out.

Input: N: Number of possible hidden states in the HMM

T: Length of the observed sequence

O: Sequence of observations ($o_1, \dots, o_T$)

A: State transition matrix, where $A[i][j] = P(q_t = j \mid q_{t-1} = i)$

B: Emission matrix, where $B[j](o_t) = P(o_t \mid q_t = j)$

Pi: Initial state distribution, $Pi[i] = P(q_1 = i)$

Other variables used in the pseudo code:

$\delta_t(j)$: stores the highest probability of any path that ends at state j at time t .

$\psi_t(j)$: stores the most likely preceding state for j at time t .

Step 1: Initialization ($t = 1$)

For each state i ($1 \leq i \leq N$):

$\delta_1(i) = Pi[i] * B[i](o_1)$

$\psi_1(i) = 0$

Step 2: Recursion ($2 \leq t \leq T$)

For each $t = 2$ to T :

For each state j ($1 \leq j \leq N$):

$\delta_t(j) = \max_{i=1 \text{ to } N} [\delta_{t-1}(i) * A[i][j]] * B[j](o_t)$

$\psi_t(j) = \operatorname{argmax}_{i=1 \text{ to } N} [\delta_{t-1}(i) * A[i][j]]$

Step 3: Termination

$P_{\text{star}} = \max_{i=1 \text{ to } N} [\delta_T(i)]$

$q_{T\text{star}} = \operatorname{argmax}_{i=1 \text{ to } N} [\delta_T(i)]$

Step 4: Path Backtracking

For $t = T-1$ down to 1 :

$q_{t\text{star}} = \psi_{t+1}(q_{t+1\text{star}})$

Output: ($q_{1\text{star}}, \dots, q_{T\text{star}}$): Most likely state sequence producing observations O

Toy Example for Viterbi

All the variables are reused and remain the same as in the Forward and Backward algorithm toy examples.

Step 1: Initialization ($t = 1$)

For each state i : $\delta_1(\text{Depressed}) = \pi_i * B_4 = 0.6 * 0.1 = 0.06$ $\delta_1(\text{Happy}) = \pi_i * B_1 = 0.4 * 0.6 = 0.24$ $\psi_1(\text{Depressed}) = \psi_1(\text{Happy}) = 0$

Step 2: Recursion

$t = 2$ ($o_2 = \text{shop}$)

For each state j :

$\delta_2(\text{Depressed})$:

Paths:

- from Depressed: $0.06 * 0.7 = 0.042$
- from Happy: $0.24 * 0.4 = 0.096$

Take max (0.096): $\delta_2(\text{Depressed}) = 0.096 * 0.4 = 0.0384$

$\psi_2(\text{Depressed}) = \text{Happy} (2)$ (since $0.096 > 0.042$)

$\delta_2(\text{Happy})$:

Paths:

- from Depressed: $0.06 * 0.3 = 0.018$
- from Happy: $0.24 * 0.6 = 0.144$

Take max (0.144): $\delta_2(\text{Happy}) = 0.144 * 0.3 = 0.0432$

$\psi_2(\text{Happy}) = \text{Happy} (2)$ (since $0.144 > 0.018$)

$t = 3$ ($o_3 = \text{nap}$)

For each state j :

$\delta_3(\text{Depressed})$:

Paths:

- from Depressed: $0.0384 * 0.7 = 0.02688$
- from Happy: $0.0432 * 0.4 = 0.01728$

Take max (0.02688): $\delta_3(\text{Depressed}) = 0.02688 * 0.5 = 0.01344$

$\psi_3(\text{Depressed}) = \text{Depressed} (1)$

$\delta_3(\text{Happy})$:

Paths:

- from Depressed: $0.0384 * 0.3 = 0.01152$
- from Happy: $0.0432 * 0.6 = 0.02592$

Take max (0.02592): $\text{delta_3}(\text{Happy}) = 0.02592 * 0.1 = 0.002592$
 $\text{psi_3}(\text{Happy}) = \text{Happy} (2)$

Step 3: Termination

$P_star = \max(\text{delta_3}(\text{Depressed}), \text{delta_3}(\text{Happy})) = \max(0.01344, 0.002592) = 0.01344$
 $q_3_star = \text{Depressed} (1)$

Step 4: Path Backtracking

$t = 2:$
 $q_2_star = \text{psi_3}(q_3_star) = \text{psi_3}(\text{Depressed}) = \text{Depressed} (1)$
 $t = 1:$
 $q_1_star = \text{psi_2}(q_2_star) = \text{psi_2}(\text{Depressed}) = \text{Happy} (2)$

The most probable state sequence is:

Time	Observation	Most probable state
1	walk	Happy
2	shop	Depressed
3	nap	Depressed

Table 1.3: Most probable sequence result for the toy example according to the Viterbi algorithm

So, with the given parameters, the most likely sequence is **Happy -> Depressed -> Depressed**.

[remainder of the page intentionally left blank]

Baum-Welch Algorithm Pseudo Code

Input:

N: Number of hidden states in HMM
M: Number of possible observations
T: Length of the observed sequence
O: Sequence of observations ($o_1, \dots, o_T$)
A: State transition matrix, $A[i][j] = P(q_t = j \mid q_{t-1} = i)$
B: Emission matrix, $B[j](k) = P(o_t = k \mid q_t = j)$
Pi: Initial state distribution, $Pi[i] = P(q_1 = i)$
(Initialize A, B, and Pi to some guess or prior)

Repeat Step 1 and 2 until convergence:

Step 1: E-step — Compute expected counts using the current parameters

1.1. Use the Forward algorithm to compute $\alpha_t(i)$:

$\alpha_1(i) = Pi[i] * B[i](o_1)$ for $1 \leq i \leq N$

For $t = 2$ to T :

For $j = 1$ to N :

$\alpha_t(j) = \sum_{i=1}^N [\alpha_{t-1}(i) * A[i][j]] * B[j](o_t)$

This calculates $\alpha_t(i)$, the probability of observing the first t values and being in state i at time t . The process recursively combines previous probabilities with transition and emission probabilities.

1.2. Use the Backward algorithm to compute $\beta_t(i)$:

$\beta_T(i) = 1$ for $1 \leq i \leq N$

For $t = T-1$ down to 1 :

For $i = 1$ to N :

$\beta_t(i) = \sum_{j=1}^N [A[i][j] * B[j](o_{t+1}) * \beta_{t+1}(j)]$

Compute $\beta_t(i)$, the probability of observing the remaining sequence after time t given the system is in state i at time t . This process is working backward from the end to the start.

1.3. Compute $\gamma_t(i)$ and $\xi_t(i,j)$:

For $t = 1$ to $T-1$, for $i = 1$ to N , for $j = 1$ to N :

$$\xi_t(i,j) = [\alpha_t(i) * A[i][j] * B[j](o_{t+1}) * \beta_{t+1}(j)] / P(O|\lambda)$$

For $t = 1$ to T , for $i = 1$ to N :

$$\gamma_t(i) = [\alpha_t(i) * \beta_t(i)] / P(O|\lambda)$$

$$\text{where } P(O|\lambda) = \sum_{i=1}^N \alpha_T(i)$$

For each time t , $\gamma_t(i)$ is the probability of being in state i at time t given the entire observation sequence. $\xi_t(i,j)$ is the probability of being in state i at time t and state j at time $t+1$ given all observed data. Both of these are calculated using the forward and backward probabilities normalized by the total likelihood $P(O|\lambda)$.

Step 2: M-step — Update the parameters

2.1. Update π :

$$\pi[i] = \gamma_1(i)$$

2.2. Update A :

For $i = 1$ to N , for $j = 1$ to N :

$$A_{\text{new}}[i][j] = \sum_{t=1}^{T-1} \xi_t(i,j) / \sum_{t=1}^{T-1} \gamma_t(i)$$

2.3. Update B :

For $j = 1$ to N , for $k = 1$ to M :

$$B_{\text{new}}[j][k] = \sum_{t: o_t = k} \gamma_t(j) / \sum_{t=1}^T \gamma_t(j)$$

End Repeat

Output:

Estimated A , B , and π maximizing $P(O|\lambda)$

Step 2 - Identify and illustrate regimes

Bull Regime - Identify and Illustrate Bull Regimes

In financial time series, the bull regime refers to a period that is characterized by a continuous upward trend in prices. In the context of HMMs, this bull regime is a hidden state in which the probability of observing increasing prices is higher. This state can also be thought of as a state of market optimism, increased demand, or other factors driving up prices.

In regime detection, we associate one of the hidden states of the HMM with the bull regime based on observed data.

Identification of the bull regime

Data transformation: The observed time series (here, the crude oil prices) are transformed into a sequence indicating price changes (e.g., 0 for decrease, 1 for neutral, 2 for increase)

HMM training: The Baum-Welch algorithm is used to train the HMM for this transformed series, and we infer sequences of hidden states that best explain the data.

Regime Attribution: Each hidden state is characterized by its emission probabilities (i.e., the likelihood of observing an increase or decrease in the given regime). The hidden state with the highest probability of observing a price increase is called the bull regime.

Illustration of a bull regime with an example

Suppose the emission probabilities for the three hidden states are as follows:

Hidden State	P(Price Up)	P(Price Down)	Interpretation
State 0 - Bear	.10	.90	Bear
State 1 - Neutral	.55	.45	Stagnant/Neutral
State 2 - Bull	.85	.15	Bull

Table 2.1: Illustration of a possible variant of emission probabilities

In Alvi's dissertation, bull regimes are detected using HMM, and the periods are identified as follows:

- The inferred hidden state aligns to a great degree with intervals of sustained price increases.

- When visualizing the time series, these periods align with the clustering of green bars and correspond with the macroeconomic environment, such as:
 - Global demand increases
 - Supply constraints
 - Geopolitical tension increases risk premium

Simplified visualization approach:

- Apply HMM to a sequence of monthly returns or price changes
- Decode the sequence of most likely hidden states
- Highlight regions classified as bull regimes.

Here is an illustrative example of a bull regime sequence:

Month	Observed change	Inferred regime
Jan	Up	Bull
Feb	Up	Bull
Mar	Down	Stagnant
Apr	Up	Bull
May	Up	Bull

Table 2.2: Illustrative example of the bull regime sequence

Bull regimes for crude oil usually happen in the following cases:

- Periods of robust, sustained growth (post-recession and post-shock recoveries, like Covid).
- Geopolitical events constraining supply (various Middle East conflicts).
- Strong speculative activity anticipating price appreciation.

[remainder of the page intentionally left blank]

Bear Regime - Identify and Illustrate Bear Regimes

In contrast to a bull regime, we talk about a bear regime when we observe a persistent and systematic downward movement in prices over a longer period. In the context of Hidden Markov Models, the bear regime is the hidden state where the probability of price decreases is the highest. (as opposed to upward moves in a bull regime). From an economic perspective, this regime reflects negative sentiment, oversupply, weakened demand, or external shocks impacting the market.

Identification of the bear regimes: The underlying method (data transformation and HMM training) remains the same as for the bull regime, except for the fact that we are looking for downward movement instead of upward movement.

Illustration of bear regimes: Here is an illustrative example of a bear regime sequence:

Month	Observed change	Inferred regime
Jan	Down	Bear
Feb	Down	Bear
Mar	Up	Stagnant
Apr	Down	Bear
May	Down	Bear

Table 2.3: Illustrative example of the bear regime sequence

If we reuse the emission probabilities from the previous chapter, then in **Table 2.1**, *State 0* is the *bear regime* because it has a very high likelihood of price decreasing (90%) and a low likelihood of increasing (10%).

Real-world examples: Bear regimes often correspond to:

- Global recessions (e.g., the 2008 2009 financial crisis or the 2020 COVID-19 shock).
- Prolonged periods of crude oil oversupply occur when OPEC increases output without sufficient demand.
- Geopolitical events that reduce confidence in the market.

It should be noted that the bear regimes are usually shorter but more intense in terms of price changes (volatility). Therefore, the bear regimes are critical from a risk management perspective.

Stagnant Regime - Identify and Illustrate Stagnant Regimes

A stagnant regime in financial time series is a period where prices display little or no movement, and neither an upward (bull) nor a downward (bear) trend dominates. In Hidden Markov Model (HMM) analysis, the stagnant regime is defined as the hidden state where the probability of observing price changes, whether up or down, is relatively balanced. This regime usually happens in case of market indecision, which is the equilibrium between supply and demand.

Identification of the stagnant regimes: The detection follows the same methodology established for bull and bear regimes, with the following difference:

- Emission probabilities: In the trained HMM, the stagnant regime is the state where the likelihood of neutral or small price changes is the greatest. And also in cases where the probabilities of up and down moves are nearly equal.
- Economic interpretation: Stagnant regimes may signal stable economic conditions, or indecisiveness in the market, or a wait-and-see attitude of the investors.

Illustration of stagnant regimes: In **Table 2.1**, *State 1* represents the stagnant/neutral regime: the emission probabilities are nearly balanced for up and down moves.

Here is an illustrative example of a stagnant regime sequence:

Month	Observed change	Inferred regime
Jan	Up	Bull
Feb	Down	Bear
Mar	Neutral	Stagnant
Apr	Up	Stagnant
May	Down	Stagnant

Table 2.4: Illustrative example of the stagnant regime sequence

Economic and market context: Stagnant regimes for crude oil prices very often happen when there is a:

- balance between supply and demand,
- uncertainty or anticipation of major policy decisions, so investors wait
- transitional period between bull and bear markets

These periods are characterized usually by lower volatility and a lack of big news impacting the fundamental underlying factors.

Step 3 - HMM updated definition

Introduction

A Hidden Markov Model (HMM) is a statistical method used to model systems that have a Markov chain as an underlying process. Additionally, they have unobserved (hidden) states, and each of these hidden states generates observed outputs (emissions) at every time step in a probabilistic way.

Definition of Hidden Markov Models

For the updated reference, we choose the Stanford University paper by **Jurafsky and Martin**.

The tuple defines a discrete-time, finite Hidden Markov Model:

$$\lambda = (\mathcal{Q}, \mathcal{V}, \boldsymbol{\pi}, \mathbf{A}, \mathbf{B})$$

where:

- $\mathcal{Q} = \{q_1, \dots, q_N\}$: Set of N possible hidden states.
- $\mathcal{V} = \{v_1, \dots, v_M\}$: Set of M possible symbols in the output (observation) alphabet.
- $\boldsymbol{\pi} = (\pi_1, \dots, \pi_N)$: Initial state distribution, where $\pi_i = P(Q_1 = q_i)$.
- $\mathbf{A} = [a_{ij}]$: State transition probability matrix, where
$$a_{ij} = P(Q_{t+1} = q_j \mid Q_t = q_i)$$
- $\mathbf{B} = [b_j(k)]$: Emission (observation) probability matrix, where
$$b_j(k) = P(O_t = v_k \mid Q_t = q_j)$$

Constraints:

$$\sum_{j=1}^N a_{ij} = 1$$
for all i (each row is a probability distribution over next states). This means that for any hidden state, the sum of the transition probabilities a_{ij} to all possible next states is always 1.

$$\sum_{k=1}^M b_j(k) = 1$$
for all j (each row is a probability distribution over possible emissions).

This means that for each hidden state, the sum of emission probabilities for all possible observed outputs is 1.

$$\sum_{i=1}^N \pi_i = 1$$

The sum of initial state probabilities for all possible starting states is also 1.

1. **Markov Property:** The hidden state sequence $\{Q_t\}$ is a first-order Markov chain:

$$P(Q_{t+1} \mid Q_1, \dots, Q_t) = P(Q_{t+1} \mid Q_t)$$

2. **Output Independence:** The observation at time t depends only on the current state:

$$P(O_t \mid Q_1, \dots, Q_t, O_1, \dots, O_{t-1}) = P(O_t \mid Q_t)$$

3. **Joint Distribution (States and Observations):**

For an observed sequence $O = (o_1, \dots, o_T)$ and corresponding hidden states $\mathbf{Q} = (q_1, \dots, q_T)$:

$$P(O, \mathbf{Q} \mid \lambda) = \pi_{q_1} b_{q_1}(o_1) \prod_{t=2}^T a_{q_{t-1}, q_t} b_{q_t}(o_t)$$

[remainder of the page intentionally left blank]

Step 4

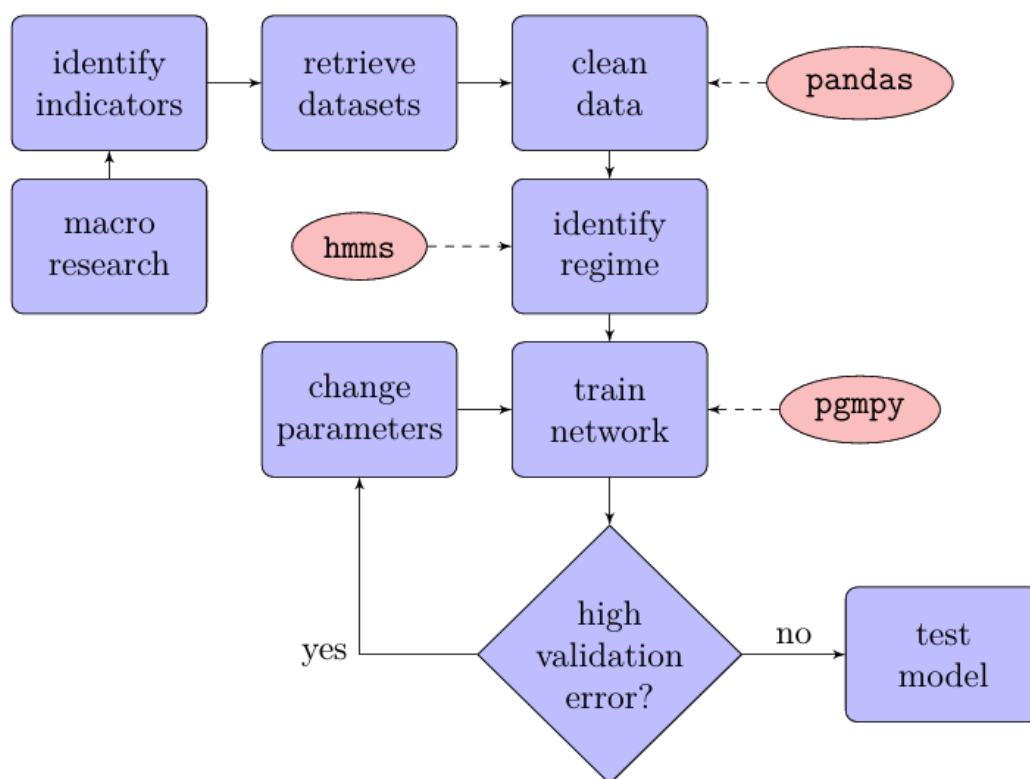


Figure 3.1: An illustration of the design process for the Crude Oil Trading model.

Source - Alvi, Danish A. "Application of Probabilistic Graphical Models in Forecasting Crude Oil Price."

4a - Macro Research, Indicator Selection, Dataset Retrieval, and Data Cleaning Framework

A vital component of the world economy, crude oil affects policy decisions, transportation, production, and inflation. The global oil market is influenced by a number of factors, including complex macro-financial relationships, exogenous shocks, and geopolitical uncertainty, in addition to traditional supply-demand fundamentals. This section looks at crude oil from a macro-financial perspective, accounting for systemic risks, global policies, and investor sentiment. We examine real-world events such as the COVID-19 pandemic, the Russia-Ukraine conflict, and changes in U.S. monetary policy to understand the factors affecting oil prices.

Geopolitical Impact (Structural Shocks and Exogenous Events)

Geopolitical unpredictability, supply chain disruptions, and war-related risks significantly impact crude oil markets. Due to significant supply disruptions and a sharp increase in crude oil prices, the Russia-Ukraine War displayed how geopolitical tensions can worsen volatility. Sudden price increases are often the result of supply uncertainty brought on by ongoing issues in oil-rich regions, such as the tensions between Iran and Israel. The U.S. Monetary Tightening, the Federal Reserve's aggressive rate hikes in response to post-pandemic inflation, stifled growth expectations and had an indirect impact on energy consumption projections.

Since oil is priced in US dollars, a stronger dollar often causes crude prices to go down because it makes it harder for foreign buyers to buy it. When the VIX goes up, it shows that the market is less willing to take risks. This happens at the same time that there is less speculation in the oil markets. A higher VIX means more uncertainty, which usually means less demand for risky assets like oil. These insights lead to the creation of a dataset with mixed frequencies and origins that includes macroeconomic, geopolitical, and market-based indicators that are important for oil pricing.

Indicator Selection

To quantify the macro-environment of oil, we selected the variables based on economic relevance, data quality, and market intuition. Our choices of variables follow three main criteria-

1. **Causally relevant** to oil market fluctuations
2. **Statistically robust** with historical coverage and frequency consistency
3. **Accessible and Reproducible**, sourced from a reputed database

Type	Abbr.	Description
Geopolitical Data	GPR	The Geopolitical Risk Index is a way to measure bad geopolitical events and the risks that come with them based on news coverage. This dataset keeps track of the overall level of geopolitical risk, which shows how tense and uncertain politics are around the world. GPR helps you figure out how world events might change the prices of goods. A high GPR could mean that the market is unstable, which could affect how you manage a portfolio or set prices for complicated derivatives.
Geopolitical Data	EUI	The Energy Uncertainty Risk Index (EUI) shows the uncertainty in energy markets driven by geopolitical and economic events. This includes government spending, tax changes, or trade policies. When EUI is high, businesses and investors are less sure about future rules, which can make them hesitant to invest.

Financial Market Data	WTI	West Texas Intermediate (WTI) is a type of crude oil and one of the three main oil price benchmarks. If you want to trade oil futures, value energy companies' stocks, or come up with ways for airlines and shipping companies to protect themselves from losses, you need WTI data.
Financial Market Data	VIX	The Volatility Index (VIX) shows the market expectations of near-term volatility, also called the "fear gauge." When VIX rises, it typically signals investor anxiety and increased expected market swings.
Financial Market Data	DXY	The US Dollar Index (DXY) indicates the strength of the US dollar against a basket of major world currencies. A stronger dollar typically suppresses oil prices. A stronger DXY can impact commodity prices and the profitability of multinational corporations.
Macroeconomic Data	GDP	Real US Gross Domestic Product (GDP) growth is an indicator of overall economic activity and health. GDP represents the total monetary value of all finished goods and services produced within a country's borders in a specific period. It's the most common measure of economic activity and health.
Macroeconomic Data	CPI	This is the US Consumer Price Index (CPI) data, showing changes in the price level of a basket of goods. Acts as a proxy for inflation. High CPI can erode purchasing power and impact asset values.
Microeconomic Data	RUR	Refinery Utilization Rate (RUR) - the percentage of total refinery capacity in use. Acts as a proxy for downstream oil demand. If RUR is high, it means refineries are busy meeting strong demand, which can support crude oil prices. For financial engineers, this data helps forecast energy market trends, gauge economic activity, and inform trading strategies for oil derivatives or energy sector investments.
Microeconomic Data	WCETUS	Weekly Crude Oil Ending Stock for the US acts as a proxy for downstream oil demand. Think of it as how much oil is sitting in storage. Financial engineers use this data to forecast energy markets and trade oil derivatives, impacting investment strategies in the energy sector.

Dataset Retrieval

We used a mix of public sources such as Yahoo Finance, FRED (Federal Reserve), EIA (Energy Information Administration), and [PolicyUncertainty.com](https://www.policyuncertainty.com). All datasets were fetched using api or standard Python libraries. All series were downsampled to a monthly frequency, either via end-of-period values or suitable statistical aggregation (mean for rates, last value for prices) methods.

Abbreviation	Frequency	Source	Start	End
GPR	Monthly	policyuncertainty.com	Jan 1985	June 2025
EUI	Monthly	policyuncertainty.com	Jan 1996	Nov 2023
WTI	Daily (resampled Monthly)	Yahoo Finance	Jan 2000	Jun 2025
VIX	Daily (resampled Monthly)	Yahoo Finance	Jan 1996	Nov 2023
DXY	Daily (resampled Monthly)	Yahoo Finance	Jan 1996	Nov 2023
GDP	Monthly	FRED	Jan 2000	Jun 2025
CPI	Monthly	FRED	Jan 2000	Jun 2025
RUR	Monthly	FRED	Jan 2000	Jun 2025
WCESTUS	Weekly (resampled Monthly)	EIA	Aug 1982	Jun 2025

We retained only the overlapping window where all variables were available, primarily 2000–2023.

Data Cleaning Framework

We have applied the following steps to clean each series to ensure robustness, non-sparse, and consistent formatting.

- A. Outlier Detection:** We employed the Interquartile range (IQR) method to identify the extreme values in the dataset. Points falling below $Q1 - 3 \times IQR$ or above $Q3 + 3 \times IQR$ were flagged. Contextually implausible outliers (e.g., >100% utilization, negative inventory levels) were removed or replaced.
- B. Missing Values and Anomalies:** Variables with negative values (such as oil inventory) that logically cannot fall below zero were eliminated. For short gaps, missing values were handled by linear interpolation, and where required, mean substitution. The average of the preceding days was used in place of the infamous April 2020 negative WTI price.
- C. Time Index Alignment:** To facilitate accurate regression and regime-switching, we standardized all time indices to a monthly frequency and ensured alignment across all features. This provides the statistical rigor and consistency in multivariate time-series modeling.

As a result, every variable has been cleansed, aligned, and anomaly-free. The series no longer contains any missing or illogical values. The final dataset supports robust statistical modeling, simulation, and causal inference.

4b - Regime Process and (Hidden Markov Models) HMM Algorithm

Regimes in Financial Time Series

Regimes are various underlying states that affect asset behavior over time in the context of financial markets. These regimes frequently show up as periods of high volatility (such as during pandemics or wars) and periods of a more peaceful economic environment, with more stable prices for crude oil. Regime-based models take these abrupt structural changes into account, in contrast to conventional linear models that presume constant relationships. In order to more accurately predict risk and return, financial engineers search for tools that enable them to recognize such shifts in market dynamics.

A time series on the price of oil is far from stable. Consider the 2020 COVID-19 oil crisis or the spike in demand following the conflict in Ukraine. These are a few instances of unique regimes that call for various approaches to investing. We can enhance derivative pricing models, optimize hedging positions, and predict portfolio-level shocks before they occur by comprehending and identifying such patterns. Regime modeling is a crucial component of strong financial engineering systems in this regard.

Hidden Markov Models (HMM)

HMM is a statistical model used to depict systems in which hidden states influence observable results. An HMM is usually represented graphically as a chain-like structure that shows the relationships between observations and hidden states over time. *The Markov property, which is the model's assumption that the system evolves through a series of hidden states, states that the probability of moving to a new hidden state solely depends on the current hidden state.* An emission probability distribution is used to generate a specific observation from the current hidden state at each time step.

The observations and the hidden states are the two main components of HMM. Hidden states are the underlying unobservable factors influencing the system. Observations are the measurable outcomes influenced by these hidden states.

1. Arrows between hidden states at different time steps, representing the state transition probabilities.
2. Arrows from hidden states to corresponding observations, representing the emission probabilities.
3. The initial state distribution, representing the probability of starting in a specific hidden state, is also a crucial element of an HMM.

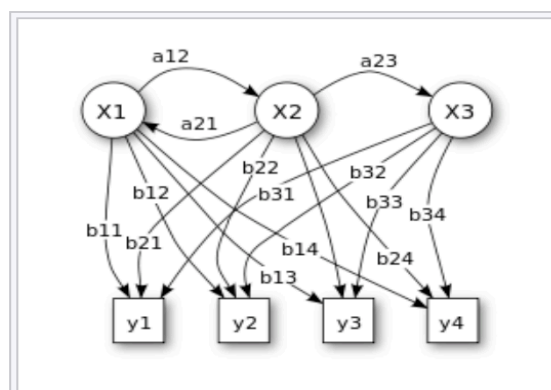


Figure 1. Probabilistic parameters of a hidden Markov model (example)

X — states
 y — possible observations
 a — state transition probabilities
 b — output probabilities

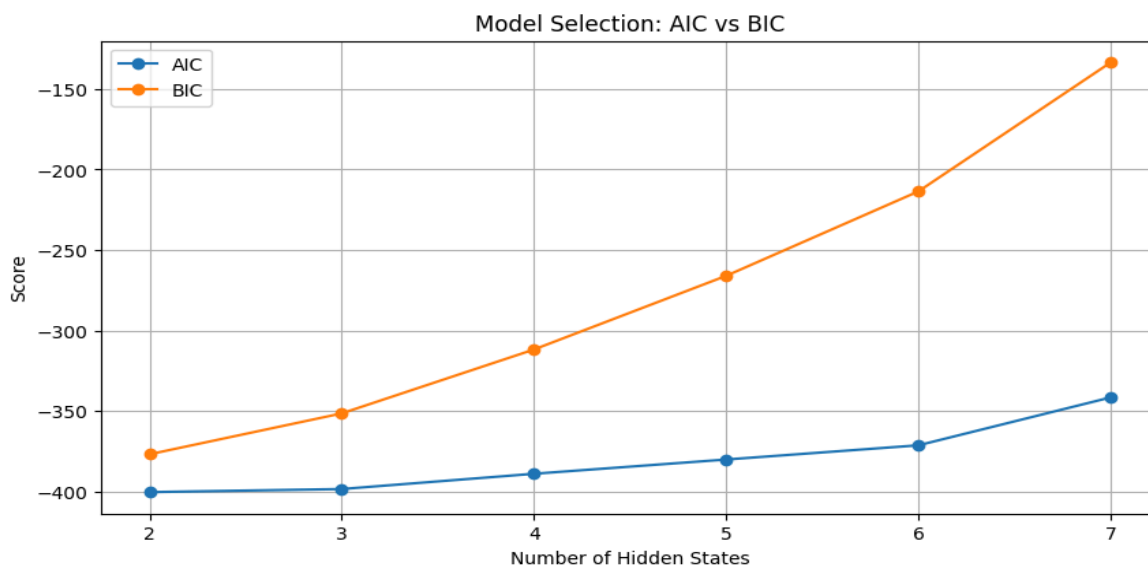
In time-series data, HMMs are especially helpful for identifying unobserved regimes. We can see the prices of crude oil, but we cannot see the low, moderate, or high volatility regime. Using probability-based learning and the observed data, HMM enables us to infer these hidden states. It allows us to map sequences where a hidden market condition drives observable behavior and acknowledges the temporal dependencies.

By revealing hidden patterns that conventional tools miss, HMMs aid in the development of more intelligent models from the standpoint of financial engineering. For instance, knowing the regime (high volatility, for example) allows you to modify position sizing, volatility surfaces, or hedge ratios appropriately when managing risk in a crude oil ETF or pricing options. It connects real-world financial applications with data science.

The model links these regimes to observable data, such as returns or volatility levels, and learns the probability of changing from one regime to another. It can calculate the likelihood of being in a particular state at a specific moment with training. Although this may seem abstract, consider it this way: an HMM is attempting to predict the weather (the regime) solely by looking at what people are wearing (the observed data). It improves over time at identifying, based only on human behavior, whether the market is sunny (bullish), cloudy (stagnant), or rainy (bearish). The model infers such regimes for oil markets based on price or volatility changes.

Application of HMM to Crude Oil Prices

We used HMMs to analyze the monthly returns of WTI crude oil prices in our project. We postulated that there are three latent regimes in which the market functions: low, medium, and high volatility. The model categorizes each period into one of these hidden states by feeding in historical returns, which allows us to examine how regimes change over time.

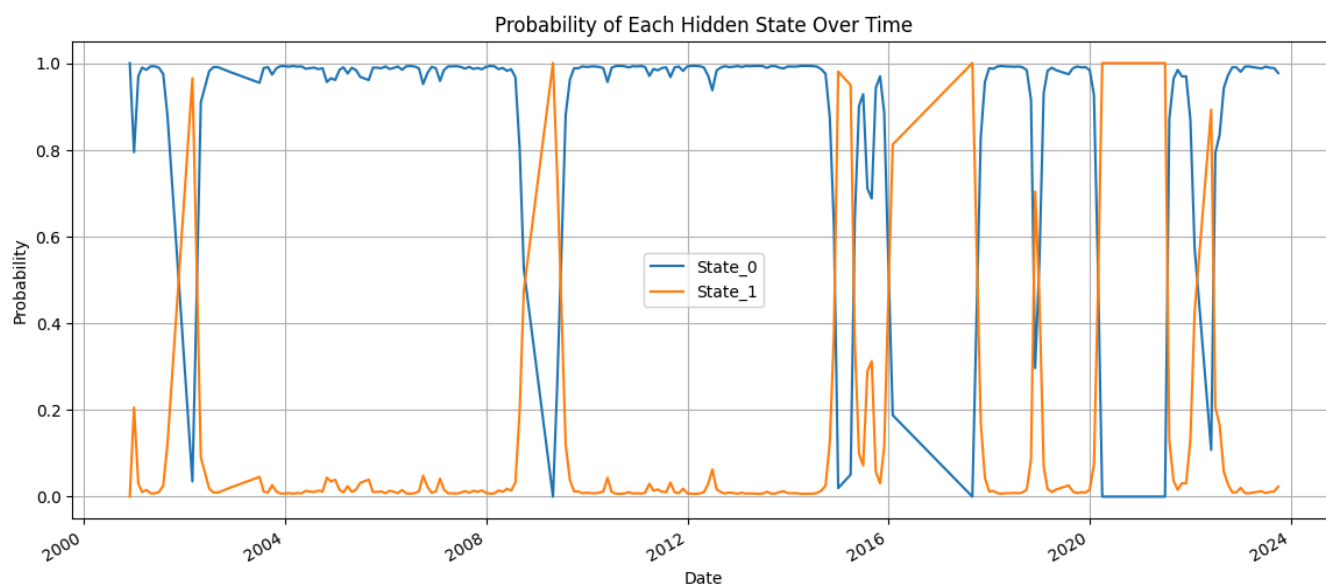


We found that stable periods in 2017–2018 corresponded to low-volatility states, whereas times such as April 2020 (during the COVID crash) matched high-volatility regimes. A two-state HMM was used to help strike a balance between explanatory power and model complexity. We also learned more about transition patterns by visualizing the odds of being in each regime. For example, the market almost never goes straight from low to high volatility without first going through an intermediate phase.

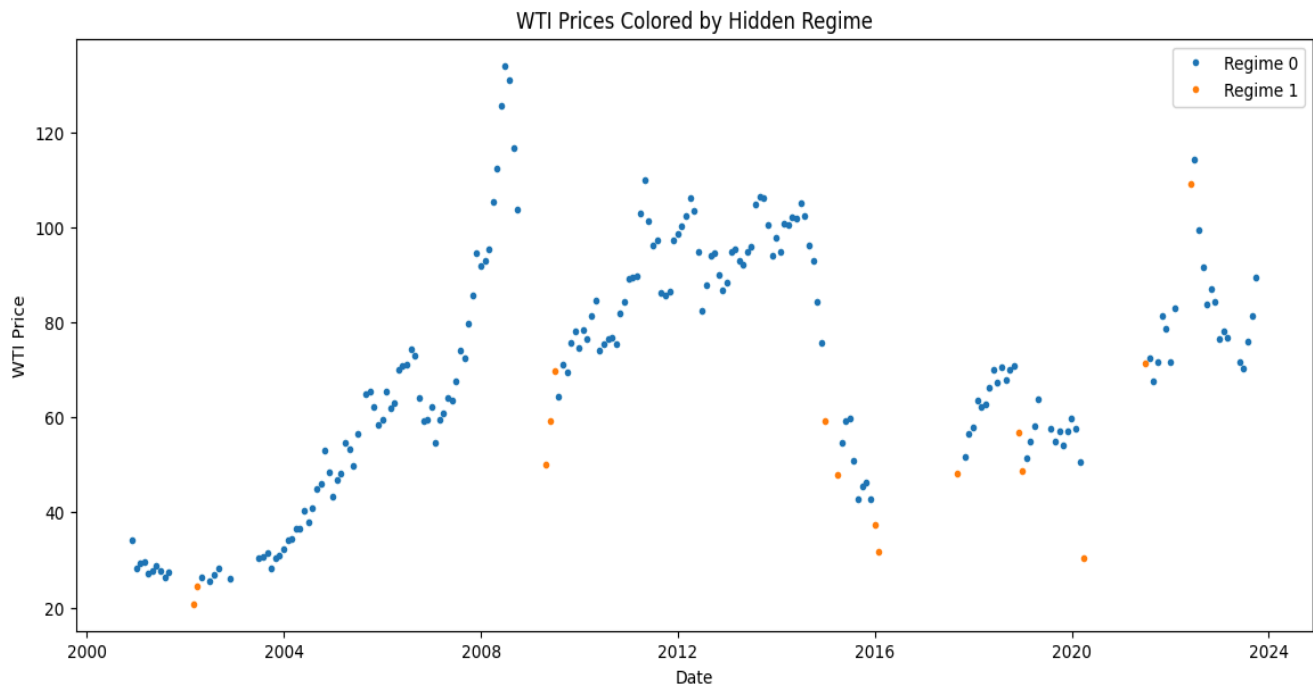
We used Python libraries like `hmmlearn` to implement our HMM. A time series of WTI log returns served as the input. After initializing the model with two hidden states, we iteratively learned the parameters that best fit the data using the Expectation-Maximization algorithm. A decoded sequence indicating the market's monthly regime was the result.

Real-World Interpretation

A probability heatmap that displayed the likelihood of each regime at each time point was one of the outputs that we plotted. The 2008 recession and the 2020 pandemic were two well-known crises that correlated with peaks in the high-volatility regime. These visualizations validated the model's ability to identify significant structure in the data. This kind of analysis can be used directly to create dynamic portfolio strategies or modify option Greeks according to the current regime.



An intuitive market condition can be mapped to each regime that the HMM has identified. For instance, a "bear market" with declining prices and increasing uncertainty could be represented by State 1. State 0 is a "bull market" with rising prices and stability. Once deciphered, these regimes assist investors in making more informed choices regarding capital allocation, leverage, and position size.



We also detected early warning signs by superimposing regime probabilities on price charts, such as when the likelihood of entering a bear regime began to increase quickly. For financial engineers who need to calibrate stress test scenarios, forecast tail risks, or create adaptive trading strategies, this level of insight is invaluable. The HMM offers a statistically based prism through which patterns in the seemingly random fluctuations of oil prices can be seen.

[remainder of the page intentionally left blank]

4c - Network Training using *pgmpy* and Parameters' Testing and Validation

Bayesian Networks for Macroeconomic Modeling

Bayesian Networks (BNs) are potent probabilistic graphical models that depict the conditional dependencies between variables with the use of directed acyclic graphs. When it comes to financial engineering, BNs enable us to simulate complex connections among market, macroeconomic, and geopolitical factors that might not be apparent in linear models. When dealing with uncertainty, incomplete data, and causal inference—all of which are common in energy markets like crude oil—these structures are particularly helpful.

In our situation, BNs are used to *visualize interdependencies and draw probabilistic conclusions*, like calculating the probability of a regime change in light of observed market conditions. BNs can assist traders, risk managers, and policymakers in more clearly simulating various economic scenarios by capturing relationships such as how refinery utilization (RUR), volatility (VIX), or inflation (CPI) affect market regime or oil returns.

Although Hidden Markov Models were useful in identifying latent regimes in oil prices, they are unable to provide insight into the ways in which other macroeconomic factors interact and impact these regimes. Bayesian Networks (BN) come in handy in this situation. A BN is a graphical model that uses a *Directed Acyclic Graph* (DAG) to represent conditional dependencies between variables. It enables us to see and measure the causal relationships between factors like market volatility (VIX), inflation (CPI), and dollar strength (DXY), as well as how they affect oil prices and the regimes that underlie them.

It is essential to understand these interdependencies in financial engineering. For example, better forecasting and stress-testing are made possible by knowing whether rising inventory levels lead to a change in the regime of the oil market or whether volatility spikes precede regime changes. BNs provide a more structured and interpretable method of identifying possible causal drivers than traditional correlation matrices, which is particularly helpful in risk analytics, algorithmic strategy design, and portfolio construction.

Training the Bayesian Network

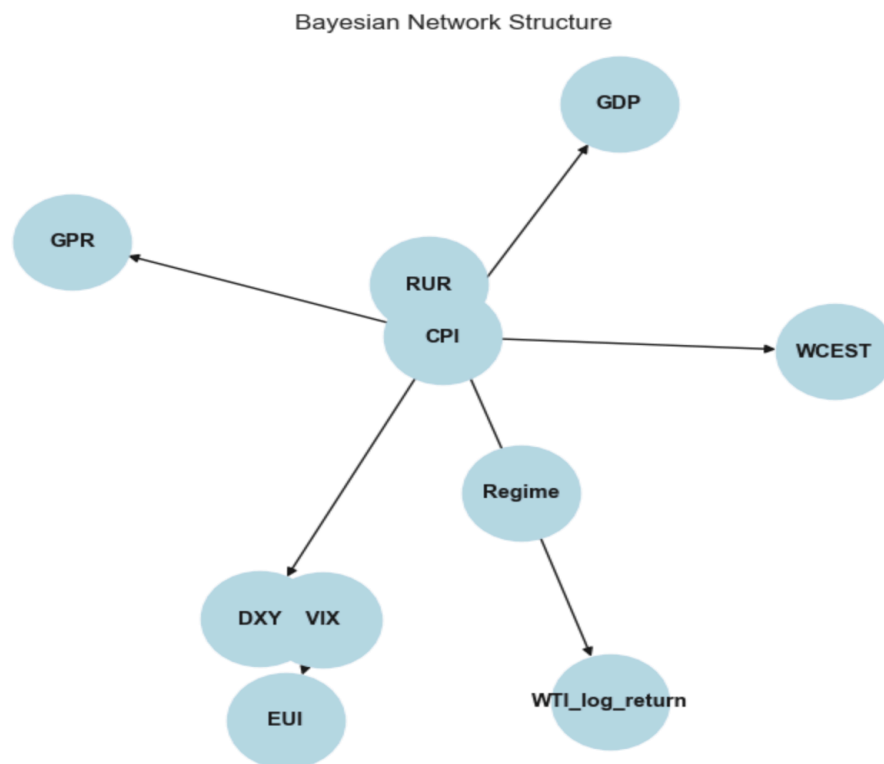
Our dataset included monthly observations from 2000 to 2023, covering key indicators like WTI prices, GPR, VIX, GDP, and CPI. The target node in our Bayesian network was the regime labels that the HMM had predicted in Step 4b. Prior to applying structure learning to determine the ideal set of edges based on scoring techniques like BIC and K2, we used *pgmpy* to discretize the continuous variables (apart from Regime) using quantile-based binning. These edges represent conditional dependencies discovered from the data. This guarantees compatibility with structure learning algorithms that use scoring functions like BIC or K2 to assess goodness of fit, like Hill Climbing.

We used the Hill Climbing algorithm with BIC scoring in a data-driven structure learning approach. This method finds the structure that best explains the data by iteratively adding, removing, or reversing network edges. The resulting graph, which can be seen in Figure: Bayesian Network Structure, showed clear and understandable relationships between the variables, such as:

- VIX and DXY were among the several nodes that CPI impacted.
- CPI and RUR had a direct effect on the regime.
- The regime was conditional and dependent on WTI_log_return.

The model is then fitted using Bayesian Estimation with BDeu priors, which provides robust handling of small data samples. In this step, each node's conditional probability distributions (CPDs), contingent on its parent nodes, were calculated. These CPDs are essential for determining the likelihood that a variable will take on a particular value given its determinants, such as forecasting high oil returns in the presence of high VIX and high CPI.

The resulting network showed some interesting relationships: for instance, CPI affects several nodes, including WCEST (inventory) and the Regime itself, whereas VIX and DXY directly affect EUI (energy uncertainty). This arrangement makes sense economically since monetary policy is influenced by inflation, which in turn affects oil demand, uncertainty, and inventory changes.



Inference and Testing the Bayesian Model

After training, we made inference-based predictions using the Bayesian Network. To find out which regime the model would most likely predict, for example, we queried it with high VIX values. The outcome was obvious: the model produced Regime 0, which denotes times of high volatility, demonstrating that the structure effectively represented the connection between market stress and changes in the price of oil.

- The probability of being in a high-volatility regime is directly impacted by the CPI and the Refinery Utilization Rate (RUR).
- Both VIX and DXY were below the CPI, illustrating how inflation affects both currency strength and uncertainty.
- The regime acts as a mediator between asset return behavior (WTI_log_return) and macroeconomic stress (CPI, RUR).

This type of network simulates causality in real-life energy markets. For instance, growing inflation frequently forces central banks to tighten policy, which in turn affects the VIX and DXY and, ultimately, the demand for and returns on commodities. Similarly, when RUR is high, it may signal strong demand, influencing both pricing and volatility regimes. These observations are helpful when creating rules for portfolio rebalancing or dynamic hedging strategies in various economic environments. Such inference is potent. It implies that the model can still produce precise probabilistic predictions about hidden variables, such as regimes or anticipated oil returns, even in the presence of incomplete knowledge of the economic environment. This is comparable to actual stress-testing situations in financial institutions, where we assess the downstream effects of assuming shocks to a few macro variables.

Validation and Real-World Interpretation

Domain expertise and empirical observation were used to validate the model's edges, particularly those that connected Regime, VIX, and CPI. Inference checks and scoring metrics like BIC, which penalize model complexity and reward data-fit, were used to assess the performance both qualitatively and quantitatively. Importantly, the network structure was not random; rather, it faithfully captured actual economic activity during events like the 2008 financial crisis and the COVID shock in 2020. This gives us confidence that the learned structure is both statistically optimal and economically sound.

Building risk-aware portfolios is aided by the BN model. An increase in the CPI and VIX, for instance, can now be interpreted not only separately but also in relation to the Regime and, consequently, crude oil returns. This makes it possible to develop regime-conditional adaptive strategies, like alternating between long and short volatility products based on deduced macro conditions. The Bayesian Network enhances the HMM by providing a more comprehensive system-level perspective of macroeconomic interaction. It serves as the causal foundation for the successful design of probabilistic simulations, proactive risk assessments, and hedging techniques.

Step 5: Summary and Interpretation of Results

We used a combination of macroeconomic analysis, regime-switching models, and causal inference tools to examine the dynamics of the crude oil market. Our goal was to comprehend how market indicators, policy choices, and global uncertainties affect oil prices not only in a linear statistical sense but also through structural regimes and interrelated economic drivers. We started with a clear, excellent dataset (cleaned in GWP-1) that covered market, macro, and geopolitical factors between 2000 and 2023. We then constructed two levels of analysis using this basis: first, we used Hidden Markov Models (HMMs) to identify the regime, and then we used Bayesian Networks for causal modeling.

Our preliminary analysis showed the significant impact that events such as the COVID-19 pandemic, the conflict between Russia and Ukraine, and the tightening of U.S. monetary policy have on oil markets. Indicators like GPR, VIX, CPI, and RUR show abrupt changes in investor sentiment, demand, and supply of oil as a result of these events. Nine important indicators were carefully chosen based on our domain expertise and their past correlation with oil prices. We made sure that the inputs to our model were stable, dependable, and consistent over time by carefully cleaning the data and aligning the frequencies. This dataset serves as the foundation for our modeling pipeline and presents a methodical approach to managing actual financial data. Based on the statistical patterns of WTI log returns, we used HMM to divide the crude oil market into different regimes. A two-state model was the best fit according to AIC/BIC criteria. These governments essentially captured:

- A state of low volatility and high return
- A state of high volatility and low return

The pre-COVID bull run, the COVID crash, and the recovery periods are all examples of real market phases that fit this regime classification well. Deeper causal analysis was made possible by the HMM's ability to translate complex return patterns into an understandable regime variable. This is particularly helpful for risk management, as knowing when you're in a risk-on or risk-off regime can help you make better capital allocation and hedging decisions.

By showing the causal relationships between various macro and market indicators and oil returns, as well as regime transitions, the Bayesian Network provided yet another level of insight. We found out that CPI and RUR primarily drove the regime. Inflation had an impact on VIX and DXY, reflecting feedback loops in markets driven by policy. The conditional dependence of WTI returns on the regime validated the predictive significance of the HMM output. We illustrated how the BN can be used for stress testing and forward-looking inference, two essential tools in any financial engineer's toolbox, using scenario analysis, such as determining the probability of being in a turbulent regime under high VIX.

Step 6

Transforming the Time Series

The paper describes how a financial time series must be transformed before it is suitable for estimation by a Hidden Markov Model (HMM). The raw time series, consisting of crude oil prices, is first converted into continuously compounded returns using the logarithmic difference formula, which is just the difference between the log of the current price and the log of the previous price at time $t-1$, and this will ensure that the data approximate stationarity. The objective of this transformation is to convert raw price data into a form that reveals the hidden regimes (market phases) that an HMM can capture. This is absolutely critical since HMMs assume that the underlying regimes are stationary processes, i.e., discrete time steps and stationarity within regimes.

Next, the returns are discretized into symbols suitable for a Discrete Time HMM. In Alvi's model, this is accomplished by setting thresholds that classify returns into states such as "up", "down", or "neutral". For instance:

- A large positive return would indicate a "bullish" regime
- A large negative return would indicate a "bearish" regime
- Near zero would indicate a "sideways" or "neutral" regime

The threshold levels are typically chosen based on the empirical quantiles or standard deviations. This converts the continuous return stream into a finite sequence of symbols, which matches the emission assumptions of the HMM.

Finally, to deal with noise and short-term fluctuations, smoothing or moving averages may be applied before discretization. This step ensures that the regime identification focuses on structural shifts rather than random daily volatility, which can occur at any point.

The resulting transformed time series is therefore a sequence of discrete symbols representing the market's behavior — ready for the HMM to learn the transitions between hidden states (regimes).

Altogether, the transformed series is now fully ready for the estimation phase, as it has been cleaned, it is stationary, and structured in a way that the HMM can model changes in underlying regimes through shifts in location or scale, such as sudden jumps in volatility.

Learning the Parameters

We must now estimate the parameters of the Hidden Markov Model, which would require us to look at the state transition probabilities, the initial probabilities, and the parameters governing the emission distributions, such as the mean and variance for each regime (assuming a Gaussian emission model). In

order to achieve this, the paper has opted for the Baum-Welch algorithm, an instance of the Expectation-Maximization (EM) algorithm for HMMs.

The most notable advantage of the Baum-Welch approach is that it can be treated as a black box, meaning that we do not need to explicitly work out the maximum likelihood estimates of all parameters, but rather follow the following steps:

- Provide the observed series
- Specify the number of hidden states
- Supply initial parameter estimates or allow the software to randomize them

The algorithm then moves on to iterate between the two main steps - Expectation and Maximization.

EXPECTATION

By using the current parameter estimates, we compute the expected values of latent variables. More specifically, this can be thought of as the probability of being in each hidden state at each time point and the probability of transitioning between states at each step, given the observed data.

MAXIMIZATION

During this step, we update the estimates of the transition probabilities and the initial state probabilities, as well as the emission distribution parameters, in order to maximize the expected complete-data log-likelihood calculated in the Expectation step.

We then iterate between these two steps until it converges and reaches a local maximum, which is evident through the almost negligible improvement in the log-likelihood.

We need to be wary of the fact that we are guaranteed to reach a maximum, but it may not be local, so it can be useful to run the algorithm a few times with different initial seeds and take the ones with the highest likelihood.

Once we have reached convergence, we will have the maximum likelihood estimates of the transition matrix, the initial state distribution, and the parameters for each state's emission distribution.

Finding the most likely Sequence of Hidden States

Once the HMM parameters have been estimated, the next task is to decode the hidden regimes that generated the observed data. We have to find the most probable sequence of hidden states that generated these specific emissions. The dissertation has opted for the Viterbi algorithm, which will be treated as a black box.

To summarize it, the Viterbi algorithm solves an optimization problem. It finds a unique single sequence with the highest possible joint probability out of all sequences of hidden states, given the estimated transition matrix, emission distributions, and the actual observations.

The algorithm follows the following steps:

- Recursively computes the highest probability path for each state at each time step
- Records both the highest probability and the “backpointer” (which states the optimal path came from)
- Once the end of the series is reached, the algorithm goes back through the backpointers in order to recover and re-create the most likely path of hidden states.

This all results in a single sequence of hidden states, representing the inferred regime at each time step, such as periods of high to low volatility.

This is crucial so that we can practically interpret the HMM, specifically:

- When and what shifts occur
- Typical characteristics of each regime
- Analyze the regimes and compare them to economic events and shifts in the market

By treating the Viterbi algorithm as a black box, we are able to easily obtain a sequence of hidden states, which allows us to practically reveal hidden structures and patterns that would be incredibly hard to detect otherwise. This highlights the value of the Hidden Markov Model and its practical uses compared to simpler time series models that would completely fail to capture any structural breaks.

Step 7

What are Latent Regimes?

The simplest way that this can be described is as an underlying economic or market condition that cannot be directly observed but is inferred through its effect on the observed data. In the case of the dissertation, an underlying phase of the crude oil market that cannot be directly observed but can be inferred from the pattern of returns.

Interpreting the Hidden States

BULL MARKET REGIME

Typically, we see a high probability of emitting “up” symbols (as expected), combined with a very low probability of large negative returns, and it often occurs for prolonged periods. Overall, a stable upward trend with high demand is expected, meaning that the market is favourable towards rising oil prices. If

a decode sequence shows this regime, we can expect continued price increases over a long period, so that we can enter long positions.

BEAR MARKET REGIME

Typically, this regime exhibits a high probability of emitting “down” symbols, coupled with a higher standard deviation (and therefore variance), further reflecting the uncertainty in the market. Bear periods may be shorter than bull periods, but much more volatile. This signals declining prices and oversupply in the market; we should therefore prepare for periods of elevated downside risk.

VOLATILE REGIME

During this regime, we see emission probabilities that are quite evenly spread out between the “up”, “down”, and “neutral” signals. We may also experience higher variance due to the bridging period between the bull and bear regimes. Overall, this period reflects market indecision, high speculation periods, or periods during global (political) events. We should closely analyze it in order to see the first signs of either a bear or a bull market and act accordingly during this transition period.

Putting it all together

In order to identify the latent meaning behind each hidden state, we must focus on three key parts:

- An emission matrix, which indicates which symbols are more likely to appear in given states
- Transition matrix, which shows how persistent the given state is
- Decoded sequence that connects the regimes with actual price movements

Step 8, 9

Based on the dissertation, the Hill Climb search is for learning the structure of a Bayesian Belief Network. Here, the hidden states refer to latent relationships. For example, nodes represent crude oil spot price, GDP growth, futures prices, etc. Edges represent the causal dependency between nodes.

From the learned structure, we know that a direct edge implies that one variable directly affects another one, but conditional on its parents. This hidden dependency structure is a representation of the macroeconomic relationships.

On the other hand, by using HMMs too, we get the HMM hidden states described above:

- Bull market regime
- Bear market regime
- Volatile regime

The Probabilistic Graphical Model for oil price forecasting combines both the Hill Climb search, which reveals the hidden relationships, and the HMM hidden states, allowing for a clear market regime to be defined.

What we can infer from our results (as seen below) is that in our simulated data:

- B is a hidden macro driver that influences both A and C, since it is marginally balanced
- E is another hidden input that drives D.
- B and E can be seen as root factors.
- A, C, and D are response variables.

This shows the latent conditional dependencies in the model.

In a real oil model:

- B could be global GDP growth → affects demand (A) and inventories (C).
- E could be the OPEC production plan → affects actual supply (D).

Learned structure edges:

```
[('B', 'A'), ('B', 'C'), ('E', 'D')]
```

Nodes:

```
['B', 'A', 'C', 'E', 'D']
```

Edges:

```
[('B', 'A'), ('B', 'C'), ('E', 'D')]
```

Conditional Probability Distributions:

```
+-----+-----+
| B(0) | 0.50199 |
+-----+-----+
| B(1) | 0.49801 |
+-----+-----+
+-----+-----+-----+-----+-----+
| B      | B(0)                                | B(1)                                |
+-----+-----+-----+-----+-----+
| A(0) | 0.902378592666006 | 0.1003996003996004 |
+-----+-----+-----+-----+-----+
| A(1) | 0.09762140733399405 | 0.8996003996003996 |
```

+-----+-----+-----+-----+			
+-----+-----+-----+-----+			
B	B(0)	B(1)	
+-----+-----+-----+-----+			
C(0)	0.9083250743310208	0.11438561438561438	
+-----+-----+-----+-----+			
C(1)	0.09167492566897918	0.8856143856143857	
+-----+-----+-----+-----+			
+-----+-----+			
E(0)	0.510945		
+-----+-----+			
E(1)	0.489055		
+-----+-----+			
+-----+-----+-----+-----+			
E	E(0)	E(1)	
+-----+-----+-----+-----+			
D(0)	0.9079844206426485	0.09816887080366225	
+-----+-----+-----+-----+			
D(1)	0.09201557935735151	0.9018311291963378	
+-----+-----+-----+-----+			

[remainder of the page intentionally left blank]

References

1. Alvi, A. Danish *Application of Probabilistic Graphical Models in Forecasting Crude Oil Price*. Dissertation. 2018, University College London, <https://arxiv.org/abs/1804.10869>
2. Jurafsky, Daniel, and James H. Martin. *Speech and Language Processing*. 3rd draft, 2024, Appendix A: Hidden Markov Models. <https://web.stanford.edu/~jurafsky/slp3/A.pdf>
3. MScFE 660 Risk Management- Module 5, Lesson 1-4: “Bayesian Networks for Financial Risk Management.” MScFE, WQU, 2024.
4. MScFE 660 Risk Management- Module 6, Lesson 1-4: “Network Learning for Risk Management.” MScFE, WQU, 2024.